



山东大学
SHANDONG UNIVERSITY

网络安全创新创业实践课实验报告

全同态密码与应用（作业 5）

第 41 组

2026 年 7 月 26 日

小组成员及分工

本实验由小组成员共同完成，各成员主要负责内容如下：

表 1: 小组成员及分工

姓名	学号	班级	主要负责内容	对应章节
姜良萍（组长）	202300460098	网安 1 班	负责实验整体方案设计与工程环境搭建；完成 OpenFHE 环境配置、CKKS 参数初始化、密钥生成以及实验工程框架设计，负责实验代码整合与整体流程维护。	第 1 章；第 3.1 节；第 4.1 节
王奕文	202300460056	密码 1 班	负责密文卷积方案设计与理论分析；完成二维卷积计算分析、CKKS 槽打包方案设计、卷积窗口与槽位偏移映射推导，以及密文卷积计算原理设计。	第 2.1 节；第 2.2 节；第 2.3 节
李雨晴	202300460025	网安 2 班	负责明文参考计算与实验结果验证；完成明文卷积程序设计、测试数据准备、实验结果整理、误差计算以及正确性验证分析。	第 3.3 节；第 5 章；第 6 章
曾静雯	202300460045	网安 1 班	负责密文卷积核心实现；完成输入编码与加密、密文旋转、密文与明文权重乘法、密文累加以及程序流程整合测试。	第 3.4 节；第 4.2 节；第 4.3 节

目录

1	实验目的	4
2	实验环境	4
3	实验原理	5
3.1	二维卷积计算	5
3.2	CKKS 同态加密与槽打包	7
3.3	基于旋转与明文乘法的密文卷积	10
3.4	输出槽位与正确性验证	13
4	实验设计与实现	14
4.1	总体设计	14
4.2	实验数据与参数设置	16
4.3	明文卷积实现	17
4.4	密文卷积实现	18
5	实验结果与分析	22
5.1	卷积计算结果	22
5.2	槽位布局验证	24
5.3	正确性验证与误差分析	25
6	总结	25

1 实验目的

本实验选用开源全同态加密库 OpenFHE，基于 CKKS 同态加密方案，实现单输入通道、单输出通道条件下的二维密文卷积。实验输入为 4×4 矩阵，卷积核大小为 3×3 ，卷积步长为 1，且不进行填充，因此输出特征图大小为 2×2 。

实验采用 CKKS 槽打包编码方式，将输入矩阵按行展开并编码至同一密文的多个槽位中。在密文状态下，通过槽位旋转、密文与明文权重逐槽乘法以及密文累加，完成卷积运算。随后对密文结果进行解密，并与普通明文卷积结果进行比较，从而验证密文卷积实现的正确性。

本实验的主要目标如下：

1. 熟悉 OpenFHE 中 CKKS 上下文创建、密钥生成、编码、加密与解密的基本流程；
2. 掌握利用 SIMD 槽打包方式表示二维输入数据的方法；
3. 理解二维卷积到槽位旋转、明文乘法和密文累加操作的映射关系；
4. 实现 4×4 输入与 3×3 卷积核的密文卷积；
5. 通过误差分析验证解密结果与明文卷积结果的一致性。

2 实验环境

本实验在 Windows Subsystem for Linux 2 (WSL2) 环境中完成，具体软硬件配置如表 2 所示。

表 2: 实验环境配置

项目	配置
操作系统	Ubuntu 22.04.2 LTS
运行环境	WSL2
Linux 内核	6.18.33.2-microsoft-standard-WSL2
系统架构	x86_64
处理器	13th Gen Intel Core i9-13900H
WSL2 可用处理器资源	2 核 4 线程
内存	23 GiB
Swap 空间	32 GiB
编译器	GCC/G++ 11.4.0
构建工具	GNU Make 4.3
编程语言标准	C++17
全同态加密库	OpenFHE 1.5.1
同态加密方案	CKKS
执行平台	CPU
CKKS Batch Size	16
Ring Dimension	16384

实验程序使用 C++17 编写, 并通过 GNU Make 进行编译和构建。OpenFHE 安装于 `/home/makabaka/opt/openfhe` 目录, 实验采用其 CKKS 方案完成实数槽位编码、密钥生成、数据加密、密文旋转、密文与明文乘法、密文累加以及结果解密等操作。

实验过程中未使用 GPU, 所有同态加密运算均在 CPU 上完成。

3 实验原理

3.1 二维卷积计算

本实验考虑单输入通道、单输出通道条件下的二维卷积。输入特征图大小为 4×4 , 卷积核大小为 3×3 , 卷积步长为 $s = 1$, 填充大小为 $p = 0$ 。输入矩阵 X 和卷积核 K 分别表示为

$$X = \begin{bmatrix} x_{00} & x_{01} & x_{02} & x_{03} \\ x_{10} & x_{11} & x_{12} & x_{13} \\ x_{20} & x_{21} & x_{22} & x_{23} \\ x_{30} & x_{31} & x_{32} & x_{33} \end{bmatrix}, \quad K = \begin{bmatrix} k_{00} & k_{01} & k_{02} \\ k_{10} & k_{11} & k_{12} \\ k_{20} & k_{21} & k_{22} \end{bmatrix}.$$

对于输入高度 H_{in} 、输入宽度 W_{in} 、卷积核高度 H_k 、卷积核宽度 W_k 、步长 s 和填

充 p ，二维卷积的输出尺寸为

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - H_k}{s} \right\rfloor + 1,$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p - W_k}{s} \right\rfloor + 1.$$

将本实验参数

$$H_{\text{in}} = W_{\text{in}} = 4, \quad H_k = W_k = 3, \quad s = 1, \quad p = 0$$

代入上述公式，可得

$$H_{\text{out}} = W_{\text{out}} = \frac{4-3}{1} + 1 = 2.$$

因此，卷积输出为一个 2×2 矩阵：

$$Y = \begin{bmatrix} y_{00} & y_{01} \\ y_{10} & y_{11} \end{bmatrix}.$$

本实验采用深度学习框架中常用的二维互相关定义，即在计算过程中不对卷积核进行 180° 翻转。输出元素的计算公式为

$$y_{i,j} = \sum_{r=0}^2 \sum_{c=0}^2 x_{i+r,j+c} k_{r,c}, \quad i, j \in \{0, 1\}.$$

四个输出元素分别为

$$\begin{aligned} y_{00} &= x_{00}k_{00} + x_{01}k_{01} + x_{02}k_{02} \\ &\quad + x_{10}k_{10} + x_{11}k_{11} + x_{12}k_{12} \\ &\quad + x_{20}k_{20} + x_{21}k_{21} + x_{22}k_{22}, \end{aligned}$$

$$\begin{aligned} y_{01} &= x_{01}k_{00} + x_{02}k_{01} + x_{03}k_{02} \\ &\quad + x_{11}k_{10} + x_{12}k_{11} + x_{13}k_{12} \\ &\quad + x_{21}k_{20} + x_{22}k_{21} + x_{23}k_{22}, \end{aligned}$$

$$\begin{aligned} y_{10} &= x_{10}k_{00} + x_{11}k_{01} + x_{12}k_{02} \\ &\quad + x_{20}k_{10} + x_{21}k_{11} + x_{22}k_{12} \\ &\quad + x_{30}k_{20} + x_{31}k_{21} + x_{32}k_{22}, \end{aligned}$$

$$\begin{aligned} y_{11} &= x_{11}k_{00} + x_{12}k_{01} + x_{13}k_{02} \\ &\quad + x_{21}k_{10} + x_{22}k_{11} + x_{23}k_{12} \\ &\quad + x_{31}k_{20} + x_{32}k_{21} + x_{33}k_{22}. \end{aligned}$$

因此，实验中的二维卷积可分解为四个 3×3 滑动窗口与卷积核之间的内积。后续密文卷积需要在不解密输入数据的条件下，等价实现上述四个窗口内积。

3.2 CKKS 同态加密与槽打包

3.2.1 CKKS 的基本特点

OpenFHE 支持 BFV、BGV 和 CKKS 等同态加密方案，其主要区别如表 3 所示。

表 3: BFV、BGV 与 CKKS 方案对比			
方案	数据类型	计算精度	典型应用
BFV	整数	精确	整数计算
BGV	整数	精确	算术电路
CKKS	实数或复数	近似	机器学习与数值计算

CKKS 的同态计算关系可表示为

$$\text{Dec}_{sk}(\text{Eval}_f(\text{Enc}_{pk}(\mathbf{x}))) \approx f(\mathbf{x}),$$

其中，符号 $\approx$ 表示解密结果与明文计算结果之间存在较小的近似误差。

CKKS 与实验相关的主要特点包括：

- 支持实数和复数的近似同态计算；
- 支持同态加法、乘法和密文槽位旋转；
- 支持 SIMD 槽打包，可在一个密文中并行处理多个数据；
- 适合实现卷积层、全连接层等实数线性计算。

由于本实验的输入和卷积核均包含实数，且卷积主要由乘法和加法组成，因此选择 CKKS 方案。

3.2.2 CKKS 槽打包与 SIMD 计算

CKKS 支持将多个实数或复数编码到同一个明文或密文的多个逻辑槽位（slot）中，并在这些槽位上执行 SIMD 并行计算。其主要特点包括：

- 一个密文可以同时保存多个数据；
- 同态加法和乘法可在各个槽位上并行执行；
- 一次同态运算即可处理整组打包数据。

设两个槽向量分别为

$$\mathbf{z} = [z_0, z_1, \dots, z_{n-1}], \quad \mathbf{w} = [w_0, w_1, \dots, w_{n-1}],$$

则逐槽加法为

$$\mathbf{z} + \mathbf{w} = [z_0 + w_0, z_1 + w_1, \dots, z_{n-1} + w_{n-1}],$$

逐槽乘法为

$$\mathbf{z} \odot \mathbf{w} = [z_0 w_0, z_1 w_1, \dots, z_{n-1} w_{n-1}],$$

其中, $\odot$ 表示对应槽位之间的逐元素乘法。

在 OpenFHE 中, 可通过

`MakeCKKSPackedPlaintext(...)`

将多个实数编码为一个 CKKS 打包明文, 再使用公钥将其整体加密为一个密文。此后, 对该密文执行加法或明文乘法时, 各个槽位中的数据将并行参与计算, 从而提高密文运算效率。

3.2.3 本实验的输入打包方式

本实验输入为一个 4×4 矩阵。为了利用 CKKS 的 SIMD 并行能力, 将输入矩阵按照行优先顺序展开为长度为 16 的向量:

$$\mathbf{x} = [x_{00}, x_{01}, x_{02}, x_{03}, x_{10}, x_{11}, x_{12}, x_{13}, x_{20}, x_{21}, x_{22}, x_{23}, x_{30}, x_{31}, x_{32}, x_{33}].$$

输入矩阵与槽位编号之间的对应关系为

$$\begin{bmatrix} x_{00} & x_{01} & x_{02} & x_{03} \\ x_{10} & x_{11} & x_{12} & x_{13} \\ x_{20} & x_{21} & x_{22} & x_{23} \\ x_{30} & x_{31} & x_{32} & x_{33} \end{bmatrix} \longrightarrow \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{bmatrix}.$$

因此, 各输入元素与逻辑槽位之间满足

$$\begin{aligned} \text{slot } 0 &\leftrightarrow x_{00}, \\ \text{slot } 1 &\leftrightarrow x_{01}, \\ &\vdots \\ \text{slot } 15 &\leftrightarrow x_{33}. \end{aligned}$$

输入矩阵的编码和加密过程可表示为

$$pt_X = \text{Encode}([x_{00}, x_{01}, \dots, x_{33}]),$$

$$ct_X = \text{Enc}_{pk}(pt_X),$$

其中, pt_X 表示 CKKS 打包明文, ct_X 表示加密后的输入密文。

本实验运行时 OpenFHE 自动选取的环维数为

$$N = 16384,$$

并将 Batch Size 设置为 16。

因此, 实验只使用前 16 个逻辑槽位保存 4×4 输入矩阵。环维数主要影响同态加密方案的安全性、模数容量和计算开销, 而 Batch Size 决定本实验实际参与逻辑运算的槽位数量。

3.2.4 槽编码与系数编码的选择

CKKS 中常见的数据表示方式包括槽编码和系数编码。两种编码方式的主要区别如表 4 所示。

表 4: 槽编码与系数编码对比

对比项	槽编码	系数编码
数据表示	每个输入元素对应一个独立的逻辑槽位	每个输入元素对应明文多项式的一个系数
乘法语义	对应槽位之间逐元素相乘	多项式乘法会使不同系数相互混合
数据重排	可通过密文旋转重新排列槽位	通常通过多项式乘法或系数移位实现
并行方式	支持 SIMD 并行计算	主要利用多项式环上的运算结构
适用场景	向量运算、矩阵运算和基于旋转的卷积	基于多项式乘法的卷积和特定编码算法

在槽编码方式下, 槽向量

$$[m_0, m_1, \dots, m_{15}]$$

与明文权重向量

$$[w_0, w_1, \dots, w_{15}]$$

进行逐槽乘法后得到

$$[m_0w_0, m_1w_1, \dots, m_{15}w_{15}].$$

系数编码则将数据嵌入明文多项式的系数中：

$$m(X) = m_0 + m_1X + \dots + m_{15}X^{15}.$$

本实验选择槽编码，主要原因如下：

- 每个输入矩阵元素能够独立映射到一个逻辑槽位，数据布局清晰；
- 密文与明文乘法能够在各个槽位上并行执行；
- 密文旋转可以将卷积窗口中的不同元素对齐到指定输出位置；
- 能够直接实现

打包 $\rightarrow$ 旋转 $\rightarrow$ 逐槽乘法 $\rightarrow$ 累加

的密文卷积流程；

- 与本实验采用的 OpenFHE CKKS 卷积实现方式一致。

因此，本实验采用 CKKS Packed Slot Encoding，将输入矩阵按行打包到同一个密文的逻辑槽位中，而不采用系数编码。

3.3 基于旋转与明文乘法的密文卷积

3.3.1 卷积位置与槽位偏移

本实验将 4×4 输入矩阵按照行优先顺序打包到 CKKS 槽位中。由于输入矩阵每行包含 4 个元素，卷积核位置 (r, c) 在一维槽位布局中对应的偏移量为

$$\delta_{r,c} = 4r + c, \quad r, c \in \{0, 1, 2\}.$$

因此， 3×3 卷积核的九个位置对应的槽位偏移为

$$\begin{bmatrix} 0 & 1 & 2 \\ 4 & 5 & 6 \\ 8 & 9 & 10 \end{bmatrix},$$

即

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

其中，偏移 1 表示访问当前位置右侧的输入元素，偏移 4 表示访问下一行同一列的输入元素，偏移 5 表示访问下一行右侧一个位置的输入元素。通过对输入密文执行相应的槽位旋转，可以将卷积窗口中不同位置的元素对齐到统一的输出槽位。

3.3.2 明文权重向量与有效槽掩码

卷积输出大小为 2×2 ，四个有效输出位置对应原输入布局中的槽位

$$P = \{0, 1, 4, 5\}.$$

因此，定义有效槽掩码

$$M = [1, 1, 0, 0, 1, 1, 0, 0, 0, \dots, 0].$$

对于卷积核中的每个系数 $k_{r,c}$ ，构造对应的明文权重向量

$$W_{r,c} = k_{r,c}M.$$

例如，卷积核系数 $k_{0,1}$ 对应的明文权重向量为

$$W_{0,1} = [k_{0,1}, k_{0,1}, 0, 0, k_{0,1}, k_{0,1}, 0, 0, 0, \dots, 0].$$

该权重向量一方面在有效输出槽位中保留相应卷积核系数，另一方面将其他槽位置零，从而避免无效位置和循环旋转产生的数据对最终结果造成影响。

3.3.3 密文卷积计算过程

设输入矩阵打包并加密后得到密文

$$ct_X = \text{Enc}_{pk}(pt_X).$$

对于卷积核位置 (r, c) ，首先将输入密文旋转 $\delta_{r,c} = 4r + c$ 个槽位：

$$ct_{r,c} = \text{Rot}_{4r+c}(ct_X).$$

随后，将旋转后的密文与对应的明文权重向量逐槽相乘：

$$ct'_{r,c} = ct_{r,c} \odot pt(W_{r,c}),$$

其中， $pt(W_{r,c})$ 表示由明文权重向量 $W_{r,c}$ 编码得到的 CKKS 明文， $\odot$ 表示密文与明文之间的逐槽乘法。

将九个卷积核位置产生的密文贡献进行累加，可得到最终密文输出：

$$ct_Y = \sum_{r=0}^2 \sum_{c=0}^2 \text{Rot}_{4r+c}(ct_X) \odot pt(W_{r,c}).$$

展开后可写为

$$\begin{aligned} ct_Y = & ct_X \odot pt(W_{0,0}) \\ & + \text{Rot}_1(ct_X) \odot pt(W_{0,1}) \\ & + \text{Rot}_2(ct_X) \odot pt(W_{0,2}) \\ & + \text{Rot}_4(ct_X) \odot pt(W_{1,0}) \\ & + \text{Rot}_5(ct_X) \odot pt(W_{1,1}) \\ & + \text{Rot}_6(ct_X) \odot pt(W_{1,2}) \\ & + \text{Rot}_8(ct_X) \odot pt(W_{2,0}) \\ & + \text{Rot}_9(ct_X) \odot pt(W_{2,1}) \\ & + \text{Rot}_{10}(ct_X) \odot pt(W_{2,2}). \end{aligned}$$

输入矩阵始终以密文形式参与计算，卷积核权重保持为明文。整个密文卷积过程只使用密文旋转、密文与明文乘法以及密文加法，不涉及密文与密文乘法，因此无需执行重线性化；同时，由于计算深度较低，也不需要进行 Bootstrapping。

3.3.4 循环旋转与无效数据处理

CKKS 的槽位旋转属于循环旋转。当槽位中的元素移动超过边界时，末尾元素会循环回到起始位置。但普通二维卷积不允许输入矩阵最后一列的数据跨行参与下一行计算，也不允许最后一行的数据循环回到第一行。

为消除循环旋转带来的无效数据，本实验在每个明文权重向量中加入有效槽掩码

$$M = [1, 1, 0, 0, 1, 1, 0, 0, 0, \dots, 0].$$

旋转后的密文与权重明文逐槽相乘后，只有槽位 0、1、4 和 5 中的数据被保留，其余槽位均被乘以 0。因此，循环回绕和非法卷积窗口产生的数据不会进入最终结果。

采用的旋转方向满足

$$\text{Rot}_d(ct_X)[p] = ct_X[p + d],$$

即原槽位 $p+d$ 中的数据在旋转后被移动到槽位 p 。在此定义下，偏移集合 $\{0, 1, 2, 4, 5, 6, 8, 9, 10\}$ 能够分别将卷积窗口中的九个输入位置对齐到四个有效输出槽位。

3.4 输出槽位与正确性验证

3.4.1 输出槽位布局与结果提取

本实验的卷积输出大小为 2×2 。四个卷积窗口的左上角位置分别为

$$(0, 0), \quad (0, 1), \quad (1, 0), \quad (1, 1).$$

在输入矩阵按行打包的槽位布局中, 上述位置对应的槽位编号为

$$P = \{0, 1, 4, 5\}.$$

因此, 密文卷积结果解密后的槽位布局应为

$$[y_{00}, y_{01}, 0, 0, y_{10}, y_{11}, 0, 0, 0, \dots, 0].$$

程序从解密后的槽向量中提取四个有效输出:

$$\begin{aligned} y_{00}^{\text{FHE}} &= \text{slot}[0], \\ y_{01}^{\text{FHE}} &= \text{slot}[1], \\ y_{10}^{\text{FHE}} &= \text{slot}[4], \\ y_{11}^{\text{FHE}} &= \text{slot}[5]. \end{aligned}$$

其余槽位在明文权重掩码的作用下理论上应为 0。由于 CKKS 属于近似同态加密方案, 实际解密结果中这些槽位可能表现为绝对值极小的非零数, 但不影响有效输出的提取。

3.4.2 明文参考结果

为了验证密文卷积的正确性, 实验首先使用普通 C++ 程序计算相同输入和卷积核下的明文卷积结果:

$$Y^{\text{plain}} = \begin{bmatrix} y_{00}^{\text{plain}} & y_{01}^{\text{plain}} \\ y_{10}^{\text{plain}} & y_{11}^{\text{plain}} \end{bmatrix}.$$

随后, 对密文卷积结果进行解密和解码, 并从槽位 0、1、4 和 5 中提取结果, 得到

$$Y^{\text{FHE}} = \begin{bmatrix} y_{00}^{\text{FHE}} & y_{01}^{\text{FHE}} \\ y_{10}^{\text{FHE}} & y_{11}^{\text{FHE}} \end{bmatrix}.$$

明文卷积与密文卷积采用完全相同的输入矩阵、卷积核、步长和填充参数, 因此明文结果可作为密文计算结果的参考值。

3.4.3 误差度量与正确性判定

由于 CKKS 采用近似数值计算，解密结果可能与明文计算结果之间存在微小误差，因此本实验采用最大绝对误差评价结果正确性：

$$e_{\max} = \max_{i,j} \left| y_{i,j}^{\text{FHE}} - y_{i,j}^{\text{plain}} \right|.$$

本实验设置误差阈值为

$$\varepsilon = 10^{-5}.$$

当满足

$$e_{\max} < \varepsilon$$

时，认为密文卷积结果与明文卷积结果一致，正确性验证结果为

$$\text{Verification} = \text{PASS}.$$

否则，验证结果为

$$\text{Verification} = \text{FAIL}.$$

误差主要来源于 CKKS 编码过程中的缩放与取整、同态运算中的近似误差以及解码阶段的浮点舍入。只要误差低于设定阈值，即可认为密文卷积实现正确。

4 实验设计与实现

4.1 总体设计

本实验基于 OpenFHE 全同态加密库，采用 CKKS 同态加密方案实现单输入单输出条件下的二维密文卷积。实验整体流程如图 1 所示。



图 1: 密文卷积实验总体流程

整个密文卷积的数据处理流程可以表示为

$$X \rightarrow \text{Encode} \rightarrow \text{Encrypt} \rightarrow \text{FHEConv} \rightarrow \text{Decrypt} \rightarrow Y,$$

其中, X 表示输入矩阵, Encode 表示 CKKS 槽打包编码, Encrypt 表示输入加密, FHEConv 表示在密文状态下执行卷积计算, Decrypt 表示结果解密, Y 表示最终卷积输出。

在数据表示方面, 本实验将 4×4 输入矩阵按照行优先顺序展开为长度为 16 的向量, 并编码到同一个 CKKS 密文的多个槽位中。卷积计算完成后, 有效输出结果保存在槽位 0、1、4 和 5 中, 对应输出矩阵中的四个元素:

$$Y = \begin{bmatrix} y_{00} & y_{01} \\ y_{10} & y_{11} \end{bmatrix}.$$

具体实现过程中，首先使用普通明文卷积计算得到参考结果，随后完成 CKKS 参数初始化、密钥生成、输入编码与加密，并在密文状态下依次执行旋转、密文与明文权重逐槽乘法以及密文累加操作。计算完成后，对输出密文进行解密，提取有效槽位中的结果，并与明文参考结果进行比较。

为了验证密文卷积实现的正确性，采用最大绝对误差作为评价指标：

$$e_{\max} = \max_{i,j} \left| y_{i,j}^{\text{FHE}} - y_{i,j}^{\text{plain}} \right|,$$

其中， $y_{i,j}^{\text{FHE}}$ 表示密文卷积解密后的结果， $y_{i,j}^{\text{plain}}$ 表示普通明文卷积结果。当满足

$$e_{\max} < 10^{-5}$$

时，认为密文卷积结果与明文计算结果一致，实验验证通过。

4.2 实验数据与参数设置

4.2.1 输入数据与卷积核

本实验采用单输入通道、单输出通道的二维卷积作为测试对象。输入特征图大小为 4×4 ，卷积核大小为 3×3 ，卷积步长设置为 1，且不进行边缘填充。因此，最终输出特征图大小为 2×2 。

实验使用的输入矩阵为

$$X = \begin{bmatrix} 1.1 & 2.2 & 3.3 & 4.4 \\ 5.5 & 6.6 & 7.7 & 8.8 \\ 9.9 & 10.1 & 11.2 & 12.3 \\ 13.4 & 14.5 & 15.6 & 16.7 \end{bmatrix}.$$

对应的卷积核设置为

$$K = \begin{bmatrix} 0.2 & -0.3 & 0.4 \\ 0.5 & 0.6 & -0.7 \\ 0.8 & -0.9 & 1.0 \end{bmatrix}.$$

该卷积核包含正数、负数以及小数权重，并且九个位置的权重均非零，能够完整覆盖 3×3 卷积中的所有计算位置，避免由于零权重导致部分卷积计算被跳过。

实验中的卷积参数配置如表 5 所示。

表 5: 卷积计算参数

参数	设置
输入尺寸	4×4
卷积核尺寸	3×3
输入通道数	1
输出通道数	1
步长	1
填充	0
输出尺寸	2×2

4.2.2 CKKS 参数设置

本实验基于 OpenFHE 实现 CKKS 同态加密计算。CKKS 参数配置如表 6 所示。

表 6: CKKS 同态加密参数

参数	设置
同态加密库	OpenFHE 1.5.1
加密方案	CKKS
安全等级	HEStd_128_classic
Multiplicative Depth	1
First Modulus Size	60 bits
Scaling Modulus Size	50 bits
Batch Size	16
Ring Dimension	16384
执行平台	CPU

其中, Batch Size 设置为 16, 用于存储按行展开后的 4×4 输入矩阵。实验中输入数据仅占用前 16 个 CKKS 槽位, 其余槽位不参与计算。

OpenFHE 根据所设置的安全等级和模数参数自动选择实际环维数, 本实验运行过程中得到的 Ring Dimension 为 16384。

4.3 明文卷积实现

为了验证密文卷积结果的正确性, 本实验首先在明文状态下实现相同参数的二维卷积, 并将其计算结果作为参考值。明文卷积采用与密文卷积相同的输入矩阵、卷积核以及计算方式:

$$Y_{\text{plain}} = \text{Conv}(X, K),$$

其中, X 表示输入矩阵, K 表示卷积核, Y_{plain} 表示明文卷积输出结果。

对于每个输出位置, 程序遍历对应的 3×3 卷积窗口, 并累加输入元素与卷积核权重的乘积。其计算流程如下:

Algorithm 1 明文二维卷积计算

Require: 输入矩阵 X , 卷积核 K

Ensure: 输出矩阵 Y_{plain}

```
1: for 每个输出位置  $(i, j)$  do
2:    $sum \leftarrow 0$ 
3:   for 每个卷积核位置  $(r, c)$  do
4:      $sum \leftarrow sum + X[i + r][j + c] \times K[r][c]$ 
5:   end for
6:    $Y_{\text{plain}}[i][j] \leftarrow sum$ 
7: end for
8: return  $Y_{\text{plain}}$ 
```

得到的明文卷积结果将在后续步骤中与密文卷积解密后的结果进行比较, 通过误差分析验证密文计算的正确性。

4.4 密文卷积实现

4.4.1 CKKS 上下文与密钥生成

本实验首先完成 CKKS 加密环境初始化, 并生成密文计算所需的相关密钥, 具体流程如下:

- 根据实验参数创建 CKKS 加密上下文, 包括安全等级、计算深度、缩放参数以及槽数量等配置;
- 开启密文计算所需功能模块, 包括:
 - 公钥加密 (PKE), 用于输入数据加密和结果解密;
 - 密钥切换 (KEYSWITCH), 用于支持密文旋转操作;
 - 层级同态计算 (LEVELEDSHE), 用于执行密文加法和乘法。
- 生成公钥 pk 和私钥 sk :
 - 公钥用于加密输入矩阵;
 - 私钥用于解密最终卷积结果。

- 根据卷积计算所需的槽位偏移生成旋转密钥：

$$\{1, 2, 4, 5, 6, 8, 9, 10\}.$$

CKKS 上下文与密钥生成流程如下：

Algorithm 2 CKKS 上下文与密钥生成

Require: CKKS 参数配置

Ensure: *CryptoContext*, *pk*, *sk*, RotationKeys

- 1: 初始化 CKKS 参数；
 - 2: 创建 CKKS 加密上下文；
 - 3: 开启 PKE、KEYSWITCH 和 LEVELEDSHE 模块；
 - 4: 生成公钥 *pk* 和私钥 *sk*；
 - 5: 根据卷积偏移生成旋转密钥；
 - 6: **return** *CryptoContext*, *pk*, *sk*, RotationKeys
-

4.4.2 输入编码与加密

在完成 CKKS 上下文和密钥生成后，需要将明文输入矩阵转换为 CKKS 密文，作为后续密文卷积计算的输入。该过程主要包括输入数据展开、CKKS 槽打包编码以及公钥加密三个步骤。

- **输入数据展开：**

本实验输入为 4×4 矩阵。为了适配 CKKS 的 SIMD 槽计算方式，将二维输入矩阵按照行优先顺序展开为长度为 16 的向量：

$$\mathbf{x} = [x_{00}, x_{01}, x_{02}, x_{03}, x_{10}, x_{11}, x_{12}, x_{13}, x_{20}, x_{21}, x_{22}, x_{23}, x_{30}, x_{31}, x_{32}, x_{33}].$$

其中，第 i 个向量元素对应 CKKS 密文中的第 i 个槽位。

- **CKKS 槽打包编码：**

将展开后的输入向量编码为 CKKS Packed Plaintext：

$$pt_X = \text{Encode}([x_{00}, x_{01}, \dots, x_{33}]).$$

通过槽打包方式，整个输入矩阵仅需要使用一个 CKKS 明文表示，后续加密后也仅生成一个输入密文。

- **输入加密：**

使用前一阶段生成的公钥 pk 对 CKKS 明文进行加密：

$$ct_X = \text{Enc}_{pk}(pt_X).$$

得到的输入密文 ct_X 将作为后续卷积计算的唯一输入。在密文卷积过程中，输入数据始终保持加密状态，计算方无需获得原始输入矩阵。

输入编码与加密流程如下：

Algorithm 3 输入编码与加密流程

Require: 输入矩阵 X ，公钥 pk

Ensure: 输入密文 ct_X

- 1: 将二维输入矩阵 X 按行展开为一维向量 $\mathbf{x}$;
 - 2: 使用 CKKS Packed Encoding 对 $\mathbf{x}$ 进行编码;
 - 3: 得到 CKKS 明文 pt_X ;
 - 4: 使用公钥 pk 加密 pt_X ;
 - 5: 得到输入密文 ct_X ;
 - 6: **return** ct_X
-

经过上述步骤后，原始输入矩阵被转换为一个 CKKS 密文。后续卷积计算仅在密文 ct_X 上执行，通过密文旋转以及密文与明文权重的乘法实现卷积操作。

4.4.3 旋转、明文乘法与累加

在获得输入密文 ct_X 后，实验在密文状态下执行卷积计算。密文卷积主要由三个步骤组成：根据卷积窗口进行密文旋转、与明文权重向量进行逐槽乘法，以及将各个卷积位置产生的贡献进行累加。

卷积窗口与旋转偏移映射 由于输入矩阵采用行优先方式存储，每一行包含 4 个元素，因此卷积核位置 (r, c) 对应的一维槽位偏移为

$$\delta_{r,c} = 4r + c.$$

对于 3×3 卷积核，其对应的旋转偏移矩阵为

$$\begin{bmatrix} 0 & 1 & 2 \\ 4 & 5 & 6 \\ 8 & 9 & 10 \end{bmatrix},$$

即需要访问的输入位置偏移集合为

$$D = \{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

通过对输入密文执行对应偏移的槽位旋转，可以将卷积窗口中的不同位置元素移动到相同的输出槽位，从而完成二维卷积到一维槽位运算的转换。

明文权重向量构造 本实验中卷积核参数保持为明文，不进行加密。对于卷积核中的每个权重 $k_{r,c}$ ，构造对应的 CKKS 明文权重向量：

$$W_{r,c} = k_{r,c}M,$$

其中， M 为有效输出槽位掩码：

$$M = [1, 1, 0, 0, 1, 1, 0, 0, 0, \dots, 0].$$

因此，明文权重向量仅在有效输出位置保留卷积核权重，其余槽位置零。例如，对于权重 $k_{0,1}$ ，对应的明文向量为

$$W_{0,1} = [k_{0,1}, k_{0,1}, 0, 0, k_{0,1}, k_{0,1}, 0, \dots, 0].$$

通过这种方式，可以消除循环旋转过程中产生的无效槽位数据。

密文旋转与逐槽乘法 对于每一个卷积核位置 (r, c) ，首先对输入密文执行槽位旋转：

$$ct_{r,c} = \text{Rot}_{4r+c}(ct_X).$$

随后，将旋转后的密文与对应的明文权重向量进行逐槽乘法：

$$ct'_{r,c} = ct_{r,c} \odot pt(W_{r,c}),$$

其中， $pt(W_{r,c})$ 表示经过 CKKS 编码后的明文权重向量， $\odot$ 表示密文与明文之间的逐槽乘法。

该过程将卷积窗口中的输入元素与对应卷积核权重进行匹配，实现单个卷积位置的计算。

密文累加生成卷积结果 完成九个卷积位置的计算后，将所有贡献进行累加，得到最终的卷积输出密文：

$$ct_Y = \sum_{r=0}^2 \sum_{c=0}^2 \text{Rot}_{4r+c}(ct_X) \odot pt(W_{r,c}).$$

展开表示为：

$$\begin{aligned}
ct_Y = & ct_X \odot pt(W_{0,0}) + Rot_1(ct_X) \odot pt(W_{0,1}) + Rot_2(ct_X) \odot pt(W_{0,2}) \\
& + Rot_4(ct_X) \odot pt(W_{1,0}) + Rot_5(ct_X) \odot pt(W_{1,1}) + Rot_6(ct_X) \odot pt(W_{1,2}) \\
& + Rot_8(ct_X) \odot pt(W_{2,0}) + Rot_9(ct_X) \odot pt(W_{2,1}) + Rot_{10}(ct_X) \odot pt(W_{2,2}).
\end{aligned}$$

密文卷积整体流程可表示为：

Algorithm 4 基于旋转与明文乘法的密文卷积

Require: 输入密文 ct_X , 卷积核 K

Ensure: 输出密文 ct_Y

- 1: 初始化输出密文 $ct_Y \leftarrow 0$;
 - 2: **for** 每个卷积核位置 (r, c) **do**
 - 3: 计算槽位偏移 $\delta = 4r + c$;
 - 4: 对输入密文执行旋转: $ct' \leftarrow Rot_\delta(ct_X)$;
 - 5: 构造对应明文权重向量 $W_{r,c}$;
 - 6: 计算贡献: $ct' \leftarrow ct' \odot pt(W_{r,c})$;
 - 7: 累加: $ct_Y \leftarrow ct_Y + ct'$;
 - 8: **end for**
 - 9: **return** ct_Y
-

通过上述过程，输入数据始终保持密文状态，计算方无需获得原始输入内容，即可完成卷积计算。最终得到的输出密文 ct_Y 将在后续步骤中进行解密，并提取有效槽位用于正确性验证。

5 实验结果与分析

5.1 卷积计算结果

完成密文卷积计算后，首先对输出密文进行解密，并提取有效输出槽位中的结果。实验同时计算对应参数下的明文卷积结果作为参考值，并将两者进行比较。

实验得到的卷积计算结果如图 2 所示。

```

OpenFHE CKKS Encrypted Convolution
=====

[1] Experiment configuration
  Scheme           : CKKS
  Input shape      : 1 x 4 x 4
  Kernel shape     : 1 x 3 x 3
  Stride / padding : 1 / 0
  Output shape     : 1 x 2 x 2
  Batch size       : 16
  Ring dimension   : 16384
  Output slots     : 0, 1, 4, 5

[2] Input data

Input matrix (4 x 4) =
  [  1.100000  2.200000  3.300000  4.400000
    5.500000  6.600000  7.700000  8.800000
    9.900000 10.100000 11.200000 12.300000
   13.400000 14.500000 15.600000 16.700000 ]

Kernel matrix (3 x 3) =
  [  0.200000 -0.300000  0.400000
    0.500000  0.600000 -0.700000
    0.800000 -0.900000  1.000000 ]

[3] Convolution results

Plaintext reference =
  [ 12.230000 13.270000
    18.640000 19.950000 ]

Decrypted FHE result =
  [ 12.230000 13.270000
    18.640000 19.950000 ]

```

图 2: 明文卷积结果与密文卷积解密结果

由图 2 可以看出，密文卷积结果经过解密后得到的输出矩阵为

$$Y_{\text{FHE}} = \begin{bmatrix} 12.23 & 13.27 \\ 18.64 & 19.95 \end{bmatrix},$$

与明文卷积参考结果

$$Y_{\text{plain}} = \begin{bmatrix} 12.23 & 13.27 \\ 18.64 & 19.95 \end{bmatrix}$$

保持一致。实验结果表明，基于 CKKS 的密文卷积能够正确实现 4×4 输入和 3×3 卷积核之间的二维卷积计算。

5.2 槽位布局验证

为了验证 CKKS 槽打包方式以及输出布局设计的正确性，实验进一步输出解密后全部槽位中的数据，并检查有效输出是否位于预设槽位。

实验结果如图 3 所示。

```
[4] Packed-slot layout
Slot indices =
  [      0      1      2      3
    4      5      6      7
    8      9     10     11
   12     13     14     15 ]

Decrypted slots =
  [ 12.230000 13.270000  0.000000  0.000000
    18.640000 19.950000  0.000000  0.000000
     0.000000  0.000000  0.000000  0.000000
     0.000000  0.000000  0.000000  0.000000 ]
```

图 3: 密文卷积结果的 CKKS 槽位布局

根据实验设计，卷积输出矩阵

$$Y = \begin{bmatrix} y_{00} & y_{01} \\ y_{10} & y_{11} \end{bmatrix}$$

分别存储于槽位：

$$\{0, 1, 4, 5\}.$$

由图 3 可知：

$$\text{slot}[0] = y_{00},$$

$$\text{slot}[1] = y_{01},$$

$$\text{slot}[4] = y_{10},$$

$$\text{slot}[5] = y_{11}.$$

其余槽位均接近于 0，说明输出槽位布局符合预期，同时验证了明文权重掩码能够有效过滤循环旋转产生的无效数据。

5.3 正确性验证与误差分析

由于 CKKS 属于近似同态加密方案，密文计算结果与明文计算结果之间允许存在一定范围内的数值误差。因此，本实验采用最大绝对误差作为正确性评价指标：

$$e_{\max} = \max_{i,j} |y_{i,j}^{\text{FHE}} - y_{i,j}^{\text{plain}}|.$$

实验正确性验证结果如图 4 所示。

```
[5] Correctness verification
Maximum absolute error : 4.12114787e-13
Error tolerance         : 1.00000000e-05
Verification           : PASS
```

图 4: 密文卷积正确性验证结果

实验测得最大绝对误差为：

$$e_{\max} = 4.12114787 \times 10^{-13},$$

设置的误差阈值为：

$$\varepsilon = 10^{-5}.$$

由于：

$$4.12114787 \times 10^{-13} < 10^{-5},$$

因此密文卷积结果满足精度要求，程序输出：

Verification = PASS.

实验误差主要来源于 CKKS 编码过程中的近似计算、同态运算中的数值误差以及解码过程中的浮点舍入误差。由于实际误差远小于设定阈值，说明正确性验证通过

6 总结

本实验基于 OpenFHE 全同态加密库，采用 CKKS 方案完成了单输入单输出条件下 4×4 输入和 3×3 卷积核的密文卷积，并实现了解密结果与明文结果的比较验证。

实验结果表明,密文卷积得到的结果与明文卷积结果一致,最大绝对误差为 $4.12114787 \times 10^{-13}$, 满足设定的误差要求, 说明所实现的密文卷积过程正确。

通过本实验, 主要有以下几点收获:

- 理解了全同态加密的基本计算流程, 掌握了如何在输入保持加密的情况下完成数据计算。
- 理解了二维卷积在 CKKS 中的实现方式, 即将输入数据进行槽打包, 并通过密文旋转、明文乘法和累加操作完成卷积计算。
- 认识到全同态加密在实际机器学习推理中仍面临计算开销较大的问题, 后续需要从数据编码、同态算子设计以及计算效率等方面进行优化。